A3-PS027-TC079

IPv6 Website and Dependency Readiness Auditor

Team name: V6ision
Members:
Name: Khushee Kundra
E-mail: kundrakhushee@gmail.com

Name: Suravi Nair
E-mail: nair.suravi@gmail.com

Mentor: Dr Vidhya K, Associate Professor, Dept. of CSE AIML, PES University
E-mail: Vidhyak@pes.edu

AIORI Hackathon 3

Proposal: October 14, 2026 · Updated: October 31, 2026

Problem Description

A website might work on IPv6 at first, but some parts of it may still fail.

- **Websites have many parts:** Images, fonts, buttons and other features often load from different servers. Not all of these servers support IPv6.
- **This can affect real users:** Someone using an IPv6-only network might open a website but find that some images, features or login functions don't work.
- **Checking the address isn't enough:** Finding an IPv6 address does not guarantee that the website and all its resources can actually load.
- **What we want to solve:** We want to build a tool that tests a website using IPv4 and IPv6 separately, checks which parts load or fail, and helps identify where the problem occurs.

Update (31 Oct): we have now built this tool. On our own test website, everything looked fine over IPv4, but 4 of its 7 parts quietly failed over IPv6 and V6Scope found every one of them.

Background

Six simple ideas behind this project

IPv4 and IPv6

IPv4 and IPv6 are different protocols for network communication. A service may support both but successful access over IPv4 does not establish that the same service is reachable over IPv6.

DNS: An address is not a guarantee

DNS maps domain names to IP addresses. A records identify IPv4 addresses while AAAA records identify IPv6 addresses. These records describe available addresses not whether a connection will succeed.

A webpage is a dependency chain

Loading a page involves more than retrieving its HTML. Scripts, stylesheets, fonts, images and API calls may depend on different hosts each with its own connectivity and configuration.

Every request has multiple failure points

A resource request involves DNS resolution, connection establishment, TLS negotiation for HTTPS and an HTTP exchange. Understanding where a request fails is essential to identifying the likely cause.

The Fallback Effect

When both protocols are available browser connection strategies may allow a page to load over IPv4 despite IPv6 problems. A successful page load alone is therefore not proof of IPv6 readiness.

Evidence Based Testing

Reliable IPv6 assessment requires evidence from actual resource requests and controlled protocol-specific tests. Comparing the results helps reveal which dependencies fail and where further investigation is needed.

Literature & RFC Review

What already exists, what is missing, and what we added

Existing work	What it does	What is missing for PS-027
SAP Webres [2]	Opens webpages in a real browser, checks IPv4/IPv6 addresses of connected servers, and generates reports.	V6Scope tests IPv4-only and IPv6-only connectivity separately and gives a detailed failure analysis.
IPvFoo [10]	Shows whether webpage resources use IPv4 or IPv6.	V6Scope combines resource discovery, separate connectivity tests and structured reports.
RFC 8200, RFC 3596 [3, 4]	Explain IPv6 and how DNS uses AAAA records to store IPv6 addresses.	An AAAA record does not prove that a server or its resources are reachable.
RFC 8305-Happy Eyeballs v2 [5]	Explains how clients can try IPv4 and IPv6 to improve connection success.	Normal browser behaviour can hide failures on IPv6-only networks.
RFC 9293, RFC 8446, RFC 9110 [6-8]	Define TCP, TLS and HTTP, which handle connections, security and web communication.	V6Scope uses test evidence from each step to identify where a resource fails.
RFC 4074 [13] (new)	Describes DNS servers that answer IPv6 (AAAA) questions incorrectly.	A broken DNS answer is not proof of “no IPv6”, so V6Scope marks it inconclusive.

What we add: V6Scope uses a real browser to discover webpage resources, compares separate IPv4 and IPv6 tests, maps dependencies and identifies likely failure stages. We validated it using a controlled website with known test results, and it matched them in every run.

Proposed Solution

We open the website using IPv4 and IPv6 separately to see what works, what breaks, and where the problem is.

Submit Enter a website URL, or paste a list of websites to test.

Discover A browser opens the page and records the images, scripts, fonts and other resources it loads.

Probe Test the discovered resources separately over IPv4 and IPv6, checking DNS, TCP, TLS and HTTP where possible.

Diagnose Compare the results to find what failed on IPv6. If the evidence is unclear, mark the result as inconclusive.

Report Show the resource dependency map, test results and downloadable CSV/JSON reports with test details.

Controlled Test Site: Expected vs Found

Step	How we break it on IPv6	Expected	Found
DNS	The domain has an IPv4 address but no IPv6 address.	FAIL_DNS	same
TCP	An IPv6 address exists, but the server cannot be reached on port 443 over IPv6.	FAIL_TCP	same
TLS	The server presents a certificate that does not match the website.	FAIL_TLS	same
HTTP	The server returns an error over IPv6 but works over IPv4.	FAIL_HTTP	same
Control	The resource loads successfully over both IPv4 and IPv6.	IPv6_OK	same

We tested the tool on a website with known issues, and it identified every problem correctly.

Project status: these features are now built and tested on our controlled test website.

Optimization Proposed

Before: today's quick DNS check. After: our auditor (design goals, now measured).

Question	Quick DNS check	V6Scope approach
What is checked?	Checks the main domain's DNS records.	The webpage and its loaded resources
Other companies servers?	Usually not checked	Found automatically by the browser
Is IPv6 working?	An AAAA record alone does not prove connectivity	Tests whether resources are reachable over IPv6
Hidden faults?	IPv6 failures may be masked by successful IPv4 connections	Revealed by testing IPv6 on its own
Why did it fail?	Not explained	The failed step (DNS / TCP / TLS / HTTP) and proof
What do we get?	A basic pass/fail result	Dependency map and CSV/JSON reports with test details
Can we trust it?	Depends on what the check actually tests	Tested against our answer-key website

How the tool decides

- Check DNS records for each resource.
- Test each resource separately over IPv4 and IPv6, and record the results.
- Identify the failed stage, if possible.
- Mark as passed, failed, or inconclusive.
- Add the results to the report.

How we measured success

- Detect all known failures in the test cases -4 of 4
- Avoid flagging resources that work correctly- 0 false alarms
- Identify the likely failure stage -4 of 4
- Results stay consistent across repeated scans- 4 of 4 runs
- Report inconclusive results honestly.

Data Flow Diagram

How V6Scope processes a website and turns its test results into a report.

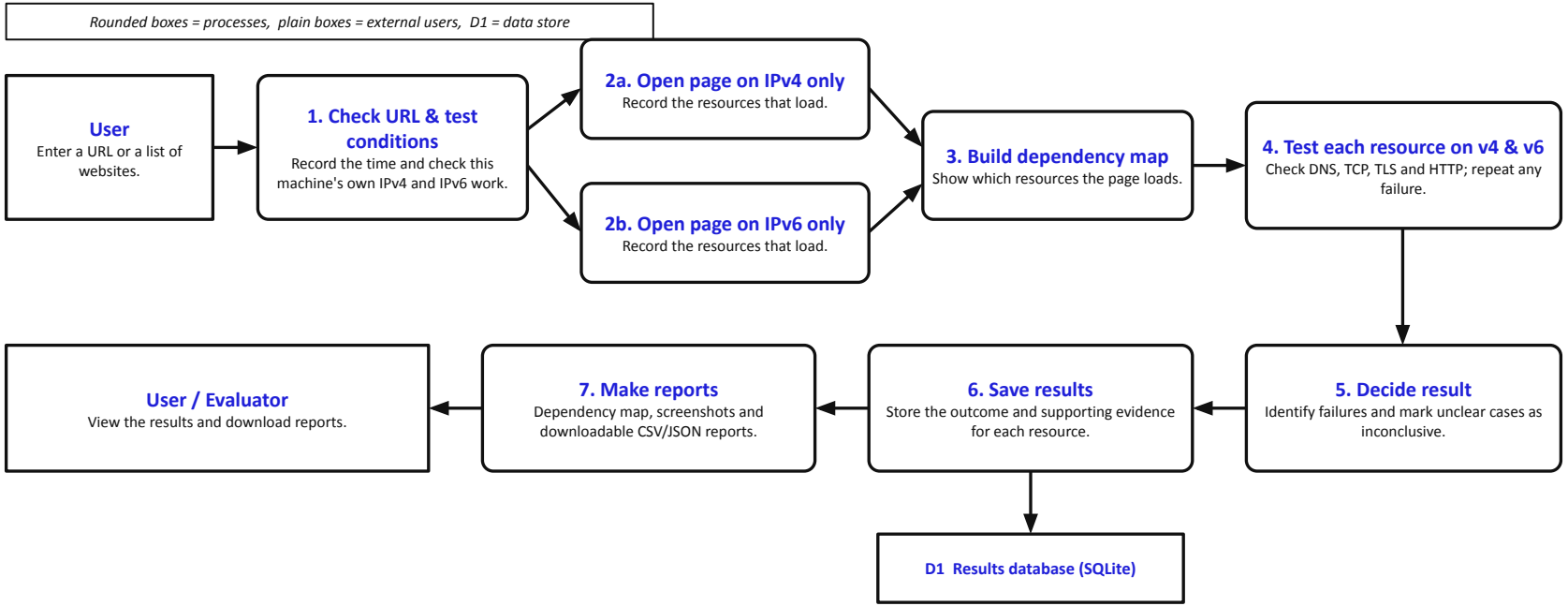


Figure: Data Flow Diagram (DFD), as built

Solution Architecture & Design

The main components of V6Scope and how they work together.

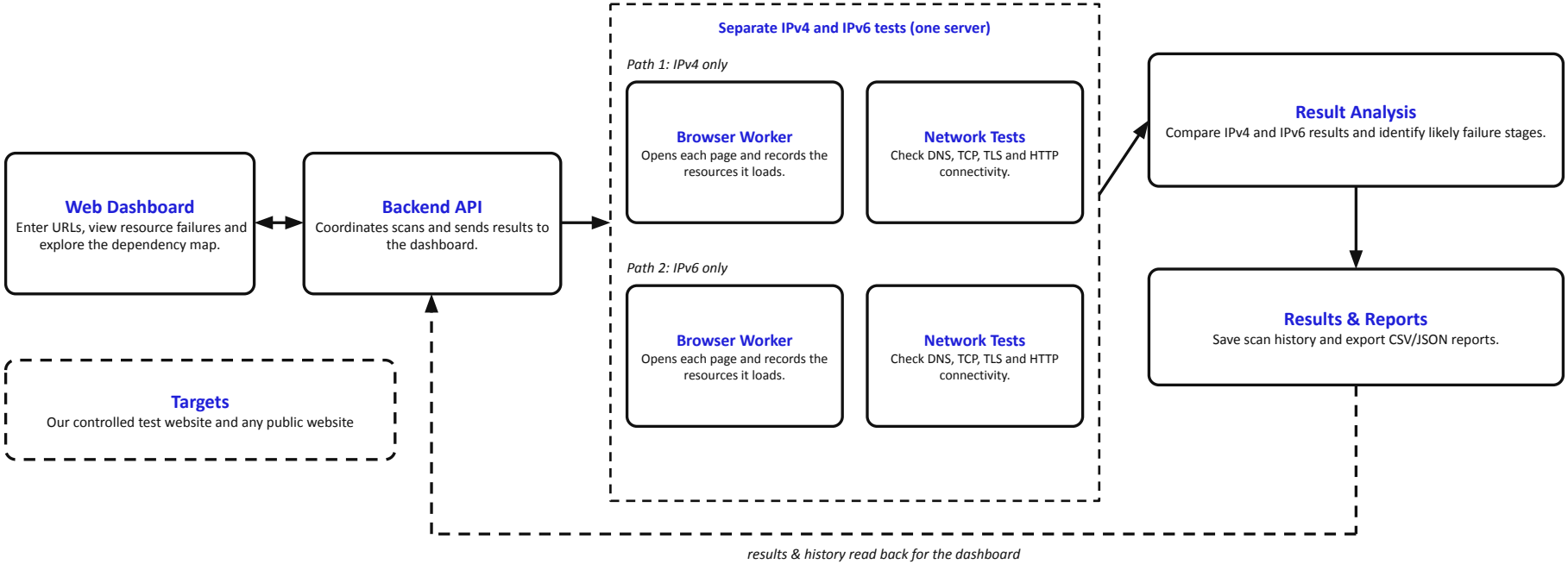


Figure: System Architecture UML Diagram (as built). Change from the proposal: one server runs both paths and forces each test to use only IPv4 or only IPv6.

Testing on our controlled website

We broke our own test website on purpose, so we knew the right answers before running V6Scope.

Case	What we broke	Right answer	V6Scope said	What the browser saw on IPv6	
C0	Nothing (main page)	IPv6_OK	IPv6_OK	200 (loaded)	✓ match
C1	Nothing (control)	IPv6_OK	IPv6_OK	200 (loaded)	✓ match
C2	No IPv6 address	FAIL_DNS	FAIL_DNS	ERR_NAME_NOT_RESOLVED	✓ match
C3	IPv6 address, nothing answering	FAIL_TCP	FAIL_TCP	ERR_CONNECTION_REFUSED	✓ match
C4	Wrong certificate on IPv6	FAIL_TLS	FAIL_TLS	ERR_CERT_COMMON_NAME_INVALID	✓ match
C5	Server error on IPv6 only	FAIL_HTTP	FAIL_HTTP	503	✓ match
C6	Broken DNS answer (honesty check)	INCONCLUSIVE	INCONCLUSIVE	ERR_NAME_NOT_RESOLVED	✓ match

7 / 7

correct in each of 4 separate runs over real IPv6

4

hidden failures: parts that work on IPv4 but break on IPv6

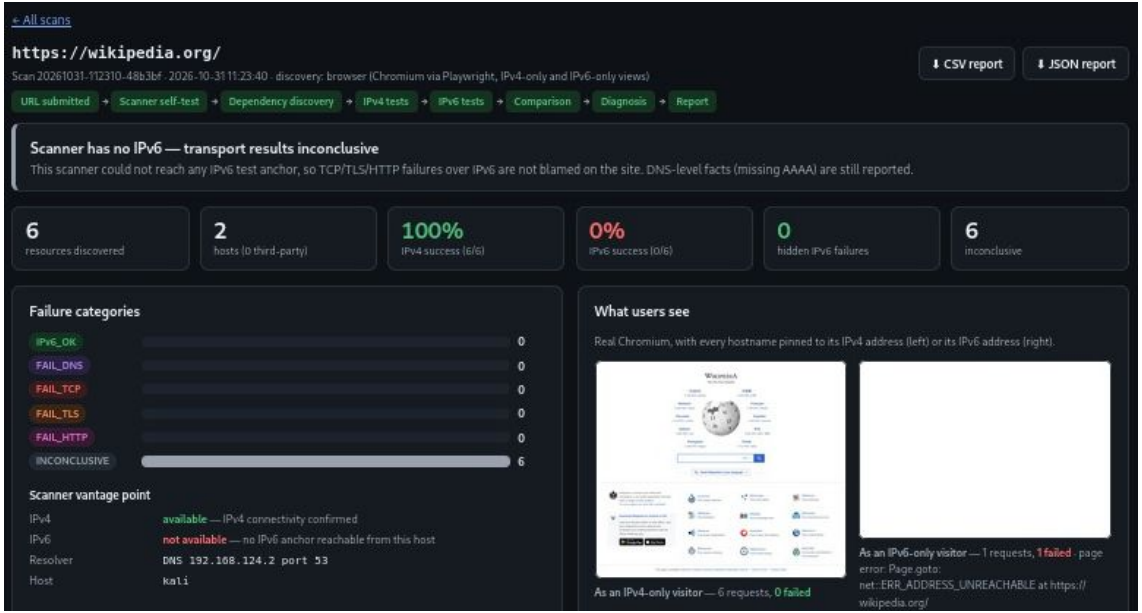
0

false alarms on the parts that work

We ran this inside a Linux virtual machine with real IPv6, because our cloud build server had IPv6 switched off.

Testing a real website: Wikipedia

Scanned by our team on a Kali Linux laptop (31 Oct 2026).



What worked

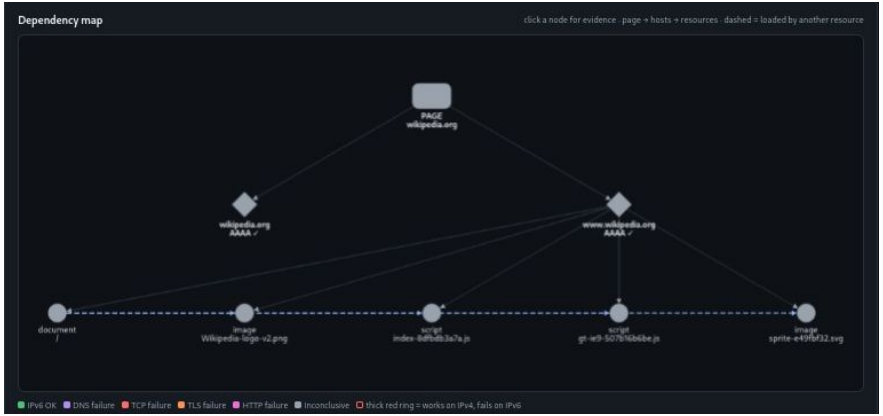
- The browser found 6 resources on 2 servers.
- IPv4: 6 out of 6 passed DNS, TCP, TLS and HTTP.
- Every server has an IPv6 (AAAA) address.

Why IPv6 says “inconclusive”

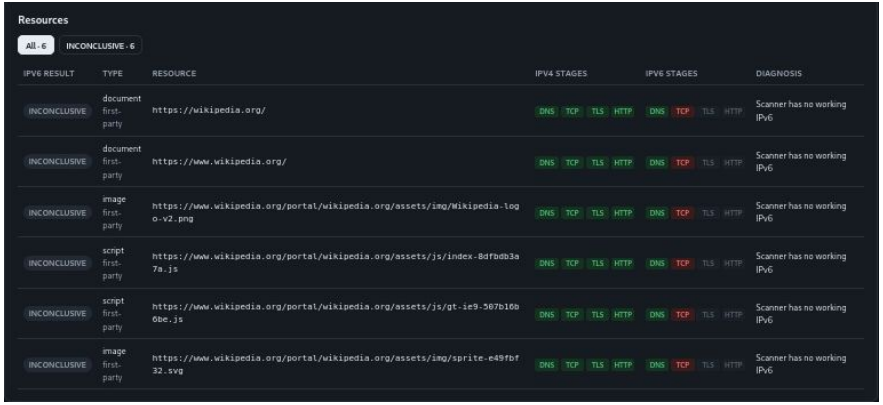
Our laptop's network had no IPv6, so every IPv6 attempt stopped at TCP. V6Scope noticed its own IPv6 was missing and did not blame Wikipedia — it marked the results inconclusive instead of failed.

Wikipedia: dependency map and results

Every resource the page loads, and how each one did over IPv4 and IPv6.



The page, its 2 servers (both have IPv6 addresses) and the 6 resources they serve.



IPv4 passes all four steps. On IPv6, DNS passes but TCP fails, because the laptop had no IPv6 connection.

Next: repeat these scans from a network with IPv6 (a mobile hotspot or our AWS server) to get real IPv6 verdicts for 30 websites.

What we built it with

The tools we used, and why.

Part	Tool	Why
Network tests	Python (socket, ssl)	Lets us force a connection to use only IPv4 or only IPv6, so nothing can quietly fall back.
DNS checks	dnspython	Tells apart “no IPv6 address”, “name doesn't exist” and “DNS server broke”.
Finding resources	Playwright + Chromium	A real browser runs the page's code, so it finds fonts and API calls a simple reader would miss.
Backend	FastAPI	Runs scans in the background while the dashboard shows progress.
Storage	SQLite	One file holds all scan history and evidence.
Dashboard	HTML, JavaScript, Cytoscape.js	Simple, works offline, draws the dependency map.
Test website	dnslib + Python HTTPS servers	Our own DNS and web servers, so every fault is exactly where we put it.
Hosting (planned)	AWS EC2 with IPv4 + IPv6	The tester itself needs real IPv6; Vercel has no IPv6.

Deployment Timeline

Milestones and methodology

Milestone	Dates (2026)	Work	Status
Level 1: proposal & setup	by 14 Oct	Problem analysis, this report, GitHub repo, team video	Done
M1: Lab & controlled site	15 - 21 Oct	Set up the IPv4 and IPv6 tests, build the answer-key website	Done
M2: Measurement engine	18 - 27 Oct	Robot browser that records every part, the 4-step tests and the decision logic	Done
M3: Dashboard & reports	24 - 31 Oct	Dependency map on the dashboard, CSV/JSON downloads with test details	Done
M4: Evaluation	28 Oct - 3 Nov	Compare results with expected outcomes, check false alarms and missed failures, repeat tests for consistency	Answer key done; 30 websites in progress
M5: Demo & refinement	31 Oct - 4 Nov	Put it online (AWS), demonstrate the tool, gather mentor feedback and make final improvements	In progress

How we work: After each milestone, we test V6Scope on our test website to make sure the results are what we expect.

Bibliography I

[1] AIORI-3 Hackathon. “PS-027: IPv6 Website and Dependency Readiness Auditor.” Problem statement, Internet Measurement category, 2026.

[2] SAP SE. “webres6: IPv6 Web Resource Checker.” GitHub, https://github.com/SAP/webres6 (accessed Oct. 14, 2026).

[3] S. Deering, R. Hinden. “Internet Protocol, Version 6 (IPv6) Specification.” RFC 8200, IETF, July 2017.

[4] S. Thomson et al. “DNS Extensions to Support IP Version 6.” RFC 3596, IETF, Oct. 2003.

[5] D. Schinazi, T. Pauly. “Happy Eyeballs Version 2: Better Connectivity Using Concurrency.” RFC 8305, IETF, Dec. 2017.

[6] W. Eddy (Ed.). “Transmission Control Protocol (TCP).” RFC 9293, IETF, Aug. 2022.

[7] E. Rescorla. “The Transport Layer Security (TLS) Protocol Version 1.3.” RFC 8446, IETF, Aug. 2018.

[8] R. Fielding, M. Nottingham, J. Reschke (Eds.). “HTTP Semantics.” RFC 9110, IETF, June 2022.

[9] W. George et al. “IPv6 Support Required for All IP-Capable Nodes.” RFC 6540, IETF, Apr. 2012.

[10] P. Marks. “IPvFoo.” GitHub, https://github.com/pmarks-net/ipvfoo (accessed Oct. 14, 2026).

[11] Microsoft. “Playwright documentation: Network.” https://playwright.dev/docs/network (accessed Oct. 14, 2026).

[12] AIORI-3 Hackathon Guidelines and Milestones. https://portal.aiori.in/aiori-3/hackathon-guideline (accessed Oct. 14, 2026).

[13] Y. Morishita, T. Jinmei. “Common Misbehavior Against DNS Queries for IPv6 Addresses.” RFC 4074, IETF, May 2005.

[14] S. Cheshire, M. Krochmal. “Special-Use Domain Names.” RFC 6761, IETF, Feb. 2013.

Thank You